

Otoroshi — Q&A

A practical, honest answer sheet for teams evaluating Otoroshi as their API Management platform. It is organized by theme rather than as a flat list of questions, so the same answer covers many of the variations a given prospect might ask.

Otoroshi is an open-source **API gateway and API Management platform** built and operated by [MAIF](#) since 2017, distributed under the Apache 2.0 license. It runs in production at MAIF, Clever Cloud and at many other organizations, and powers the commercial **Cloud APIM** platform (SaaS and on-prem with [Clever Cloud](#)) which adds hosted operations, premium support and additional enterprise extensions on top of the open-source core.

1. API lifecycle, versioning and environment promotion

Does Otoroshi manage the full lifecycle of an API (publish, update, deprecate, decommission)?

Yes. The **API** entity is a first-class object that models the complete lifecycle of a managed API:

```
staging → published → deprecated → removed
```

- **staging** — the API is being designed; only draft/test routes are served (testing mode uses a dedicated header to expose draft endpoints safely).
- **published** — the API is live; consumer plans can accept new subscriptions.
- **deprecated** — traffic still flows, but the API is flagged as on its way out; new subscriptions should not be created.
- **removed** — the API is switched off, no traffic served.

An API groups together its routes, backends, plugin flows, consumer plans, subscriptions, deployments, and developer-portal documentation under a single entity with a shared domain and context path. At runtime it is compiled down to standard routes so there is no extra overhead.

How do you manage multiple environments (staging / pre-prod / prod)?

Otoroshi is **infrastructure-as-code friendly from end to end**. Three complementary tools cover every workflow:

- **Admin REST API** — every object is a JSON resource reachable via an HTTPS API secured by API keys. Anything the UI can do, the API can do.

- **otoroshictl CLI** — the official command-line client: dump, diff, apply and sync declarative YAML/JSON files, ideal for GitOps pipelines and CI/CD.
- **Kubernetes CRDs** — a full set of Custom Resource Definitions lets you declare routes, APIs, certificates, API keys, auth modules, etc. directly in Kubernetes manifests. The Otoroshi operator / Ingress controller reconciles them into the gateway.

Typical promotion pipelines look like:

- *Git-driven*: environment configuration lives in Git, CI applies the manifests to each environment via **otoroshictl apply** or **kubectl apply**.
- *Config management*: Puppet / Salt / Ansible call the admin API or **otoroshictl** to push configuration changes.
- *Export/import*: a full datastore export/import is available, useful for cold copying an environment or bootstrapping a new one.

Development teams remain independent from ops because every change is a text artefact that can be reviewed, approved and promoted separately per environment.

Can I hot-update APIs and clients without downtime?

Yes. Otoroshi was built around **dynamic configuration**: all routes, APIs, API keys, certificates, plugins and most runtime settings can be changed at any time without restarting the gateway. Changes propagate across the cluster within seconds. There is no restart window, no "reload" command, and no request loss during on-the-fly certificate generation.

API versioning

APIs carry an explicit **version** field and keep a history of previous **versions**. A single API can expose several versions at the same time (for instance behind different context paths or headers). The developer portal (Daikoku, or the built-in pages) can surface multiple live versions of the documentation to consumers, and API deployments keep a full snapshot of what was published and when.

CI/CD for APIs

Because the whole gateway is API-first and declarative, API CI/CD is straightforward: Git PR → CI (**otoroshictl diff**) → automated apply to the target environment. The same pipeline handles plugins, certificates, auth modules, JWT verifiers, data exporters, etc.

2. Actors, roles and access to the admin plane

Roles for administrator / reference owner / developer

Otoroshi has a fine-grained RBAC model anchored on two levels:

- **Organizations** (a.k.a. tenants) — the broad isolation unit.
- **Teams** — groupings inside an organization.

Every entity lives inside exactly one organization and one or more teams.

Each admin user (or admin API key) carries a list of rights of the form:

```
[{ "tenant": "orga-prod:rw", "teams": ["team-apis:rw", "team-platform:r"]
}]
```

Access can be **r** (read), **w** (write), **not** (forbidden), with wildcards (*). This is enough to model:

- a **global super-admin**,
- a **team lead** with full control on their perimeter only,
- a **security officer** with read-only rights on everything,
- an **API owner** with rights on a given API or team only,
- a **support / analytics role** with read-only access to events and stats.

Admin API keys can be restricted in the same way via an **otoroshi-access-rights** metadata field.

SSO for the admin UI

Admin authentication is delegated to an **authentication module** that can be any of: OAuth 2.0 / 2.1 (with PKCE), OpenID Connect, LDAP (with nested groups), SAML v2, Basic, WebAuthn/FIDO2 passwordless, Auth0 passwordless, JWT, or a WASM-based custom module. Modules can be chained. Rights can be computed at login-time from IdP groups/claims.

Can I restrict an admin to just analytics?

Yes. Assign read-only rights scoped to the analytics perimeter only, and / or expose only the events / metrics part of the API via a scoped API key.

3. Multi-tenancy and governance

Can I manage several security zones from a single admin UI?

Yes. The **Organizations / Teams** model is precisely designed for that: one shared Otoroshi cluster hosting strictly isolated tenants, each with its own admins, routes, APIs, API keys and certificates. Governance can be sliced by:

- environment (staging / pre-prod / prod),
- business domain,
- organization or subsidiary,
- exposure type (internal / intra-group / external).

Isolation is enforced at both read and write level on every admin API call and UI page. Entities without explicit team assignment default to a `default` team, which keeps small deployments simple.

Multi-gateway management

A single Otoroshi control plane can drive many data planes:

- **Leader / worker clustering** — one control-plane cluster (leaders + a datastore) and an arbitrary number of data-plane workers in different zones, regions or datacenters.
- **Relay routing** — tag routes with their network location (provider, region, zone, datacenter, rack) and let leaders forward traffic to the right zone automatically, without exposing backends publicly.
- **Tunnels** — TCP/UDP/WebSocket tunnels to reach private backends without opening inbound ports.

Can gateways run as independent "micro-gateways" (mesh-style)?

Yes. Three complementary deployment patterns are supported:

- **Worker-only mode** — a gateway that only routes traffic, backed by the in-memory fast datastore, synchronized from the leader cluster.
- **Backup mode** — workers can start and keep serving traffic **even if the leader cluster is unreachable**, using the last known-good snapshot of configuration. This is explicitly designed for resilience.
- **Sidecar injection** — Otoroshi ships a sidecar deployment for Kubernetes, letting you attach a micro-gateway to a single pod for service-mesh use cases.

4. Resilience and operations

What happens if the datastore or the leader is unreachable?

Workers keep serving traffic. They cache all the configuration they need and use an on-disk backup of the last valid state, so a datastore or leader outage does not take your APIs down — only administrative write operations are suspended until the control plane recovers.

Hot patching, rolling restarts, restart with DB restore

- Configuration changes are dynamic — no full reload needed.
- The gateway is stateless from a data standpoint (no state held in JVM beyond cache), so rolling restarts are trivial: drain traffic, stop the node, restart. Clustering takes care of the rest.
- A full restart with cache purge + datastore restore follows the same pattern: restore the Redis / Postgres / Cassandra backup, start leaders first, workers resync automatically.

- `/live`, `/ready`, `/startup` and `/health` endpoints are exposed for Kubernetes liveness / readiness probes and orchestrators.

laC for the gateway itself

Otoroshi ships an official **Helm chart** and **Kustomize overlays**, plus a set of CRDs. Reference deployments are provided for AWS, Azure, GCP, Kubernetes, Docker, Clever Cloud, and bare-metal. `otoroshictl` can fully bootstrap an empty instance from a Git repository.

Upgrade / downgrade procedure

Otoroshi is **stateless at the JVM level**. The upgrade procedure is a standard rolling update:

1. Back up the datastore (Redis / PostgreSQL / Cassandra).
2. Upgrade the leader cluster first, one node at a time.
3. Then upgrade the worker cluster node by node.

Downgrades use the same pattern. Schema changes between minor versions are handled automatically; breaking changes are called out in release notes.

SLA (uptime, GTI, GTR)

The open-source distribution carries **no contractual SLA** — it is free software. Contractual SLAs (e.g. 99.5 % availability, 1 h response time, 4 h recovery time) are available as part of a **Cloud APIM support subscription**, where Cloud APIM engineers provide incident response, guaranteed reaction and repair times, and security patches.

5. Clients / consumers and the subscription flow

Client management (creation, authorizations, quotas, rotation, revocation)

The **API key** entity models machine-to-machine and developer credentials:

- `clientId` + `clientSecret` (configurable lengths) with multiple extraction methods: custom headers, HTTP Basic, Bearer token, JWT-wrapped credentials, or client-id-only.
- **Quotas**: per-second throttling, daily and monthly caps, tracked in real time and enforced cluster-wide.
- **Rate limiting**: distributed, per-IP, per-key, per-route, or custom.
- **Restrictions**: allow/deny rules on HTTP methods and paths — a single key can be scoped to `GET /v1/foo` only.
- **Expiration** (`validUntil`) and **read-only** mode.
- **Automatic secret rotation** with a grace period during which both secrets are accepted — zero-downtime credential renewal.
- Tags and metadata for routing constraints and reporting.
- **Revocation** — flip `enabled=false` or delete, effect is immediate.

Subscription flow

Two levels of subscription workflow exist:

- **Direct:** an operator creates the API key from the admin UI or API and hands it to the consumer.
- **Self-service via Daikoku** — MAIF's companion developer portal. Daikoku is a fully-featured API marketplace: catalog, search, documentation, plans with pricing and contracts, subscription requests, approval workflows, team billing, consumer analytics. It integrates natively with Otoroshi (bidirectional API key sync).

Otoroshi also ships its own developer-portal pages bound to the API entity (logo, banners, documentation pages, nav) for lighter-weight use cases that do not require Daikoku.

Internal / intra-group / external consumers are modeled via tenants, teams, tags and plans — a single API can offer different plans to different audiences with different quotas, pricing and SLAs.

Can consumers see their own stats?

Yes. Per-API-key analytics are tracked and exposed:

- In the admin UI, filtered to their keys.
- Via the analytics API (restricted by access rights).
- Via Daikoku, which ships a consumer-facing dashboard with per-plan and per-key consumption over the configured retention window (typically 1 month in hot storage, up to 1 year or more when using an external Elasticsearch / Kafka pipeline).

Access to those stats can be gated by any authentication module (OIDC, LDAP, SAML, etc.) so authorization can be centralized in an existing IdP or in an OPA-style policy engine (via the built-in Open Policy Agent WASM plugin).

6. Authentication and authorization (of end-users and machines)

Supported authentication methods

For **consumers / machines**:

- API keys (headers, Basic, Bearer, JWT-wrapped, client-id-only).
- mTLS client certificates (end-to-end, with trust store customization).
- JWT validation (multiple verifiers per route, JWE/encrypted JWT, JWT signing, re-signing, transformation).
- HMAC request signing / validation.
- Biscuit tokens (datalog-based fine-grained authorization).
- OpenFGA fine-grained authorization.

- IP allow / deny lists with CIDR, country-based (MaxMind / IPStack), time-restricted access.

For **end users**:

- OAuth 2.0 / 2.1 with PKCE.
- OpenID Connect.
- LDAP (including nested groups).
- SAML v2.
- OAuth 1.0a.
- Basic authentication.
- WebAuthn / FIDO2 passwordless.
- Auth0 passwordless.
- Otoroshi internal users store.
- WASM-based custom authentication (write it in any language).
- **Chaining** — multiple modules can be combined (e.g. LDAP + MFA).

Granularity

Every security plugin (API key, JWT, mTLS, OAuth, RBAC, Biscuit, OpenFGA, Coraza WAF, etc.) is part of the plugin chain. It can be applied:

- per gateway (global plugins),
- per API,
- per route,
- per HTTP method,
- per path / sub-path (via route matching),
- per header / query-param / cookie value.

Public vs. private path separation is first-class on every route.

Centralized input validation

Three ready-to-use validation layers exist without custom development:

- **Query-string / header / cookie validation** via built-in validators (exact, regex, wildcard, required, JSON-Path).
- **Body validation** through the JSON-Path context validator, the JQ body-filter plugin, and Coraza WAF which parses and validates request bodies against OWASP CRS rules.
- **WASM validators** — drop in any validator compiled to WebAssembly for JSON Schema, protobuf, or custom formats.

Policies and headers

- **CORS** policy per route.
- **Security headers**: HSTS, CSP, X-Frame-Options, X-XSS-Protection, X-Content-Type-Options, Referrer-Policy, Permissions-Policy.
- **WAF** — Coraza with OWASP Core Rule Set.

- **Payload size limits** — request and response body size caps, both directions (upload from client and download from provider).
 - **Bandwidth throttling** — request and response.
 - **URL allow / deny lists**, URL rewriting, path stripping, redirections.
-

7. Load balancing, routing and traffic management

Load-balancing strategies (12, not 4)

- Round Robin
- Random
- Sticky (cookie-based session affinity)
- IP Address Hash
- Best Response Time
- Weighted Best Response Time
- Least Connections
- Power of Two Random Choices
- Header Hash
- Cookie Hash
- Query Hash
- - backend failover targets

Routing criteria

- HTTP method.
- Hostname (exact, wildcard).
- Path (exact, prefix, wildcard, regex path parameters with validation).
- Header values (exact, regex, wildcard).
- Query-param values.
- Cookie values.
- Target predicates: geolocation, cloud region, datacenter, rack, zone.

Circuit breaker, retries, caching

Built-in plugins cover circuit breakers with configurable thresholds and half-open states, automatic retries with exponential backoff, response caching, gzip / brotli compression, request / response mirroring, traffic capture (GoReplay format), canary deployments (percentage-based and time-controlled), and chaos engineering (Snow Monkey — latency and failure injection).

Service discovery

DNS, Eureka (internal & external), Kubernetes API (namespace scanning), Otoroshi self-registration protocol, Tailscale.

8. Protocols supported

Protocol	Ingress	Egress
HTTP/1.1	yes	yes
HTTP/2 (incl. H2C)	yes	yes
HTTP/3 (QUIC)	yes	yes
WebSocket (with message validation, transformation, mirroring)	yes	yes
SSE (Server-Sent Events)	yes	yes
gRPC & gRPC-Web	yes (via Netty listener)	yes
GraphQL (proxy, query composition, schema-first backend)	yes	yes
REST	yes	yes
SOAP	yes (SOAP client, JSON↔XML transformation, WS-* via plugin chain)	yes
mTLS (end-to-end)	yes	yes
TCP (raw, with SNI routing)	yes	yes

For event-driven / messaging protocols (Kafka, Pulsar, RabbitMQ, MQTT, NATS...) Cloud APIM ships an **event-native extension** that translates event-bus protocols to HTTP (and vice-versa), so you can expose Kafka topics as HTTP endpoints, send requests to RabbitMQ queues, or bridge MQTT to your REST catalog — with the same plugin chain (auth, quotas, transformation, observability) as for HTTP traffic.

9. Performance and latency

Otoroshi is engineered for low overhead. On a realistic production-grade plugin chain (TLS + API key + JWT + rate limit + access log), measured overhead is typically **below 10 ms p95** on reference hardware, well under the 20 ms threshold frequently asked for in RFPs. A few context points:

- The core proxy runs on Akka HTTP (with an optional Reactor Netty server that also powers HTTP/3).
- The router uses an $O(\log n)$ prefix tree and can handle **tens of thousands of concurrent routes** without regressions.
- The data plane keeps configuration in memory, synced from the control plane.
- Plugins that dominate added latency: Coraza WAF (body inspection), WASM plugins with heavy computation, remote JWKS fetches on first hit, and external remote-call plugins (external auth, dynamic policy). All of them are individually measurable in the built-in analytics.

10. Observability and supervision

Health, metrics, logs

- `/health`, `/live`, `/ready`, `/startup` endpoints (protected by access key or API key).
- `/metrics` endpoint in **JSON** or **Prometheus** format.
- **OpenTelemetry (OTLP)** export of server metrics and server logs with gzip, gRPC and mTLS support.
- **Datadog** native exporter (logs and metrics, OTLP-compatible).
- Prometheus, StatsD, New Relic exporters.

Events and alerts

Every request, every admin action, every security event produces a structured JSON event. 25+ built-in exporters can ship those events in real time to:

Elasticsearch, Apache Kafka, Apache Pulsar, webhooks, files, S3, Mailer (Mailgun/Mailjet/Sendgrid/SMTP), console/logger, Splunk, Datadog, New Relic, GoReplay (file & S3), TCP/UDP/Syslog, JMS, WASM-based custom exporter, workflow-based custom exporter.

Alerts are standard events filtered/projected by the exporter, so your alert routing (PagerDuty, OpsGenie, Slack...) lives in your existing alerting stack.

Tracing

W3C Trace Context propagation across calls, so Otoroshi is transparent in your existing tracer (Jaeger, Zipkin, Datadog APM, Tempo, etc.).

Log redaction

Yes. The event pipeline supports projection and transformation before export, so sensitive fields (passwords, tokens, credit cards) sent as headers, query parameters, or inside the body can be stripped, hashed, or masked before logs leave the gateway. Fine-grained header/body logging is enabled per route for debugging sessions without leaking to central logs.

Security supervision

Dedicated built-in detectors for Log4Shell and React2Shell, Fail2Ban-style auto-banning, anomaly events on authentication failures, rate-limit hits, quota breaches, certificate expiration, route state for the last 90 days, security.txt (RFC 9116) and robots.txt support.

11. Certificates, TLS and PKI

- **Internal PKI**: generate CAs, sub-CAs, and leaf certificates on the fly, sign external CSRs, expose an OCSP responder and AIA endpoint, JWKS exposition for public keys, import P12 bundles.
- **ACME / Let's Encrypt** automated issuance and renewal.
- **On-the-fly certificate generation** from a CA, without request loss.
- **Dynamic TLS termination**, per-listener and per-hostname.
- **End-to-end mTLS** — both at the front and on the backend leg, with per-route trusted-CA customization.
- Kubernetes secrets ↔ Otoroshi certificates bidirectional sync.
- Tailscale certificates integration.

12. Catalog, marketplace and developer portal

Otoroshi ships:

- A **built-in developer portal** bound to the API entity — pages, navigation, logos, banners, plans, subscriptions, auto-generated credentials.
- An integration with **Daikoku** (MAIF, open-source, Apache 2.0) for the full marketplace experience: catalogs, categories, search, SLA & pricing (metered or flat), self-service subscription workflows, email notifications, team billing, consumer analytics, support of internal / intra-group / external audiences, versioning surfacing, sandbox APIs, community pages, and full contractual-exchange workflow (subscription → approval → credential delivery → usage → renewal).

Both the built-in portal and Daikoku expose OpenAPI / AsyncAPI documentation uploaded with the API definition.

13. Deployment platforms

Supported and documented deployment targets:

- **Kubernetes** — official Helm chart, Kustomize overlays, Ingress controller, CRDs, admission webhook, sidecar injection.
- **AWS** — ECS, EKS, EC2 with reference templates.
- **Azure** — AKS, App Service.
- **GCP** — GKE, Cloud Run.
- **Clever Cloud** — first-class PaaS deployment.
- **Docker / bare-metal / on-premises** — official images and systemd-ready packaging.
- **SaaS — Cloud APIM** offers a fully-managed Otoroshi with the same features plus premium extensions.

Storage backends available: Redis, PostgreSQL, Cassandra, in-memory (with file persistence), S3, HTTP.

14. Extensibility

- **200+ built-in plugins** covering auth, transformation, security, caching, compression, rate limiting, circuit breaking, transformation (jq, regex, XML/JSON, SOAP), GraphQL composition, HTML patching, redirection, mocks, maintenance/build modes, etc.
 - **WASM plugins** — write your own middleware in Rust, Go, AssemblyScript, JS, TS, Python, C, and run it sandboxed inside the gateway (Extism-based runtime).
 - **Scala plugins** — native JVM plugins packaged as jars for maximum performance and direct JVM integration.
 - **Workflows** — a visual flow engine with a step-by-step debugger for building complex processing pipelines without code.
 - **Admin extensions** — add your own entities, admin API endpoints and admin-UI pages that live alongside built-in ones.
 - **Open Policy Agent** — evaluate OPA policies directly inside the gateway (via WASM).
-

15. Security assurance and support

Security audits

Otoroshi has been deployed in production at MAIF (a top-10 French insurance company) since 2017. It has undergone internal security reviews, pen-tests, and has had public CVE responses published transparently on GitHub. Formal third-party audits and **bug-bounty programs** are part of the Cloud APIM commercial offer.

Vulnerability handling

- Disclosures are handled through the GitHub security advisories channel (private disclosure, coordinated fix, then public CVE).
- Subscribers to Cloud APIM support receive proactive notification of every advisory, a pre-release patch when possible, and a remediation window tied to severity:
 - **Critical** — patch within 24–72 h.
 - **High** — patch within 1 week.
 - **Medium** — patch within the next monthly release.
 - **Low** — patch in the next minor release.
- Anonymous usage reporting is **opt-in**.

Licensing

Otoroshi is released under **Apache 2.0**. MAIF is committed to keeping Otoroshi open-source. There is no dual-license or Business Source License trap: contributions are accepted under the Apache 2.0 CLA, and the project is driven in the open on GitHub. Cloud APIM adds commercial extensions and support on top of the unmodified open-source core.

Commercial support

Support contracts are available through **Cloud APIM** with tiered SLAs covering availability (e.g. 99.5 % and up), response time (GTI, down to 1 h), recovery time (GTR, down to 4 h), and engineering access for roadmap influence and architecture reviews. Support scope includes the OSS distribution even when self-hosted.

Release cadence and LTS

- Minor releases every 4 to 8 weeks with new features, bug fixes and security patches.
- Patch releases on demand for critical issues.
- **LTS tracks** maintained 12–24 months are available as part of the Cloud APIM support offer — production teams that do not want to track **master** can stay on an LTS branch and only apply backported patches.

References and community

Public adopters include MAIF, SNCF Connect & Tech, French public-sector entities, and many more. Community channels: GitHub, Discord. Reference calls with existing users can be arranged by Cloud APIM on request.

16. Anticipated questions about costs and "gotchas"

What is the cheapest path to production?

Self-host the OSS distribution on a small Kubernetes cluster (or 3 VMs) with PostgreSQL, one leader and two workers. This is a valid production setup for low-to-medium traffic and costs only the infrastructure you provide.

When do I actually need Cloud APIM?

- You need a contractual SLA and 24/7 incident response.
- You want the marketplace-grade event gateway (Kafka, Pulsar, RabbitMQ, MQTT → HTTP), the AI gateway, or other premium extensions.
- You want Clever Cloud engineers to run the platform for you (SaaS) instead of operating the cluster yourself.
- You want long-term LTS branches with guaranteed backport windows.

What are the required skills to operate Otoroshi?

Day-to-day operations require standard SRE / platform-engineering skills: Kubernetes (or VM) operations, PostgreSQL or Redis administration, monitoring with Prometheus/Grafana/OTEL, and basic JVM tuning. There is **no Scala knowledge required** to run or configure Otoroshi — the admin UI, API, CLI and CRDs cover everything. Writing custom plugins requires either WASM (any language) or Scala (for native plugins).

17. Quick feature matrix by profile

Profile	Primary tools
Developer (API producer)	API entity UI, otoroshictl , K8s CRDs, admin API, dev portal
Developer (API consumer)	Daikoku / developer portal, consumer analytics, sandbox APIs
API product owner	Daikoku / developer portal, subscription workflows, stats
Platform / ops admin	Admin UI, admin API, OTEL / Prometheus / Grafana, cluster view
Security officer	Read-only admin, Coraza WAF, audit events, advisories feed
Finance / billing	Daikoku billing, per-plan metering

18. In one paragraph

Otoroshi is an Apache-2.0 API gateway and API Management platform. It covers the full lifecycle of an API (design, publish, version, deprecate, decommission), supports HTTP/1/2/3, WebSocket, gRPC, GraphQL, SOAP and TCP, adds end-to-end mTLS and a first-class PKI, gives you 200+ built-in plugins plus WASM and Scala extensibility, scales horizontally via a leader/worker cluster that keeps serving even when the datastore is unreachable, offers fine-grained multi-tenant RBAC, works natively on Kubernetes / AWS / Azure / on-prem / Clever Cloud / SaaS, and is backed by a commercial offer (Cloud APIM) for SLAs, LTS, premium event and AI extensions, and managed operations.